

21CSC558J - DEEP LEARNING APPROACHES

Module-1 - Fundamentals of Deep Learning **Session-2**

Agenda

- Introduction to neural networks
- Machine learning vs deep learning
- Biological motivation of neural networks
- Fundamentals of tensor flow-data structures in tensor flow
- Perceptron model
- Perceptron learning rule
- Learning Ex-OR problem
- Activation functions
- Role of activation function
- Different types of activation functions

Backpropagation

Backpropagation

Step 1: Assign random W and B values

Step 2: Calculate final output using these values

Step 3: Estimate error using error function

Step 4: Find those W and B which can reduce this error

Step 5: Update W and B , repeat from step 2

$$w = w - \alpha \Delta w$$

$$b = b - \alpha \Delta b$$

α is learning rate, Δw and Δb are unit steps

Alpha determines number of steps we take in downward direction

Backpropagation

Example



$$w_1 x_1 + w_2 x_2 + b_1 = z$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Error Function

$$= -y \log(y') - (1 - y) \log(1 - y')$$

Backpropagation

Step 1 – Initialization

$$w1 = 2, w2 = 3, b1 = -4$$

Step 2 – Forward Propagation

$$Z = 2 \times 10 + 3 \times -4 + (-4) = 4$$

Applying activation function $\sigma(Z) = 0.982$

Step 3 – Error Calculation ($y=1, y'=0.982$)

$$E = -y \log(y') - (1-y) \log(1-y') = -1 \log(0.982) - (1-1) \log(1-0.982)$$

$$E = 0.0079$$

Backpropagation

Step 4 – Back Propagation

$$\frac{\partial E}{\partial y'} = \text{slope of error wrt } y' = \frac{\partial(-1 \times \log(y'))}{\partial y'} = -\frac{1}{y'}$$

$$\frac{\partial y'}{\partial z} = \text{slope of activation function wrt } z = \frac{e^{-z}}{(1 + e^{-z})^2}$$

$$\frac{\partial z}{\partial w_1} = x_1 = 10 \quad \frac{\partial z}{\partial w_2} = x_2 = -4 \quad \frac{\partial z}{\partial b} = 1$$

To get $\frac{\partial E}{\partial w_1}$ i.e. Δw_1 we apply chain rule $\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial y'} \times \frac{\partial y'}{\partial z} \times \frac{\partial z}{\partial w_1} = -0.186$

Similarly $\frac{\partial E}{\partial w_2} = 0.0746 \quad \frac{\partial E}{\partial b} = -0.0186$

Backpropagation

Step 4 – Updating w and b

$$w1 = w1 - \alpha \Delta w1 = 2 - 5 \times -0.186 = 2.93$$

$$w2 = w2 - \alpha \Delta w2 = 3 - 5 \times 0.0746 = 2.627$$

$$b = b - \alpha \Delta b = -4 - 5 \times -0.0186 = -3.907$$

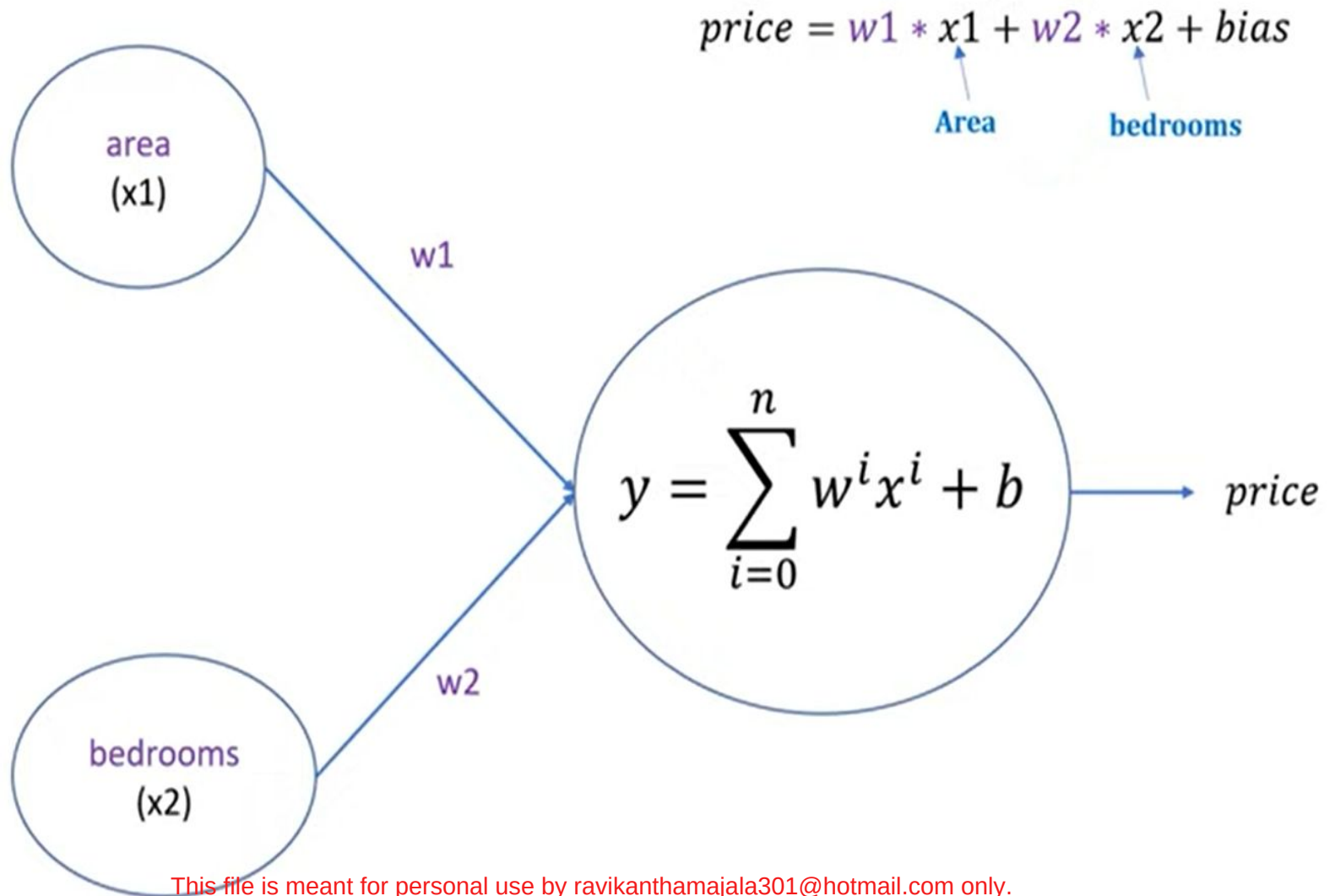
Repeat Step 2 –

$$z = 2.9 \times 10 + 2.6 \times -4 + -3.9 = 14.7$$

Applying activation function $\sigma(z) = 0.999$

Chain Rule

Chain Rule

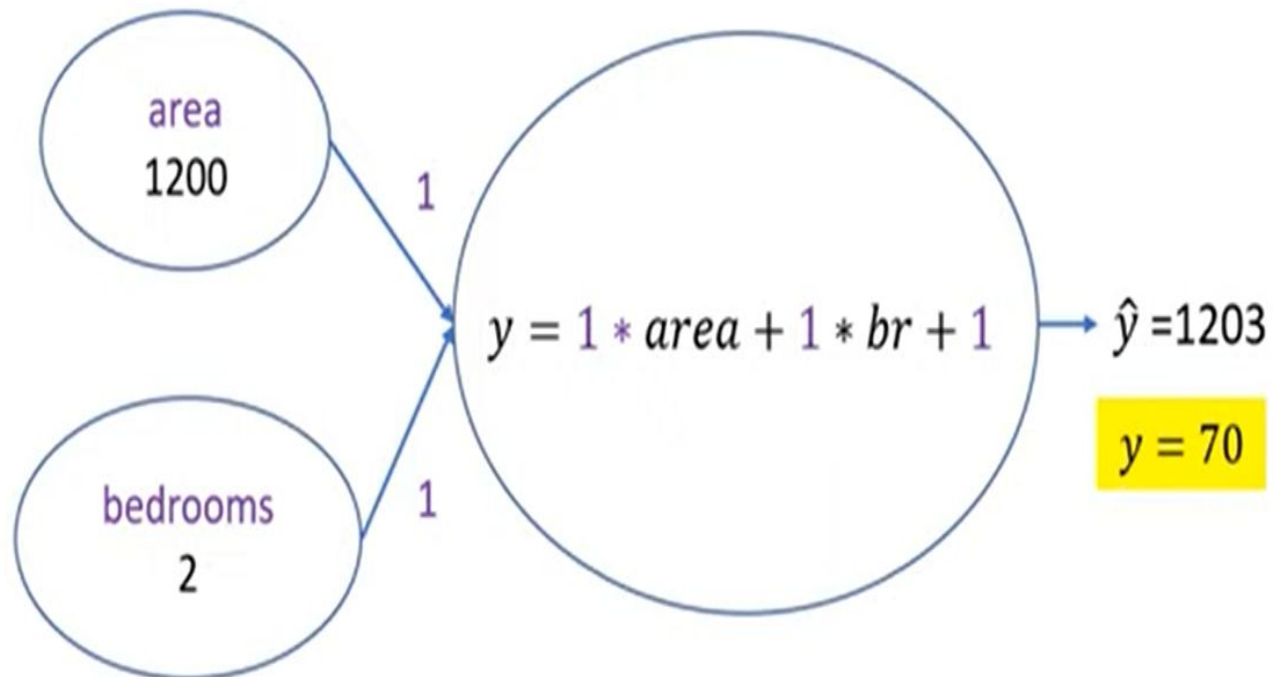


Chain Rule

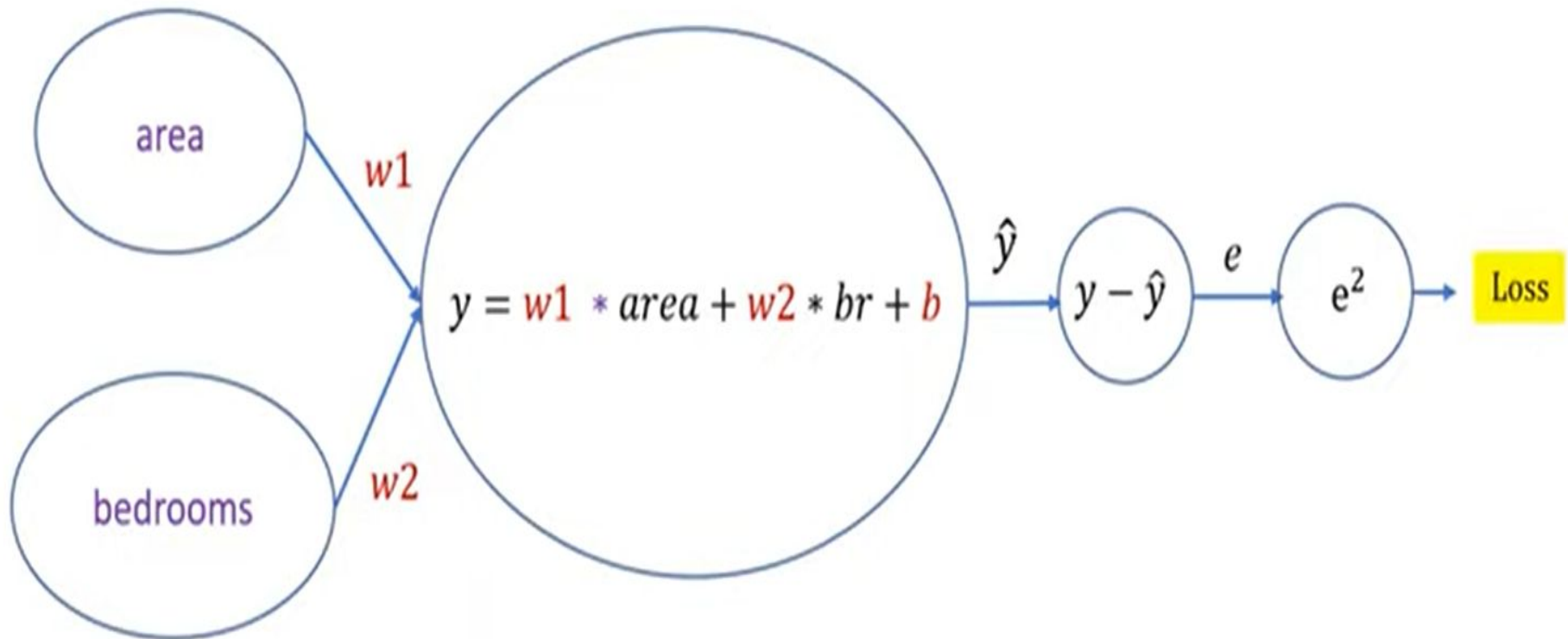
$$error1 = y - \hat{y} = 1133$$

$$squared\ error1 = 1283689$$

area	bedrooms	price
1200	2	70
1500	3	120
2100	4	230
1100	3	105
1300	2	90
900	1	55



Chain Rule



Chain Rule

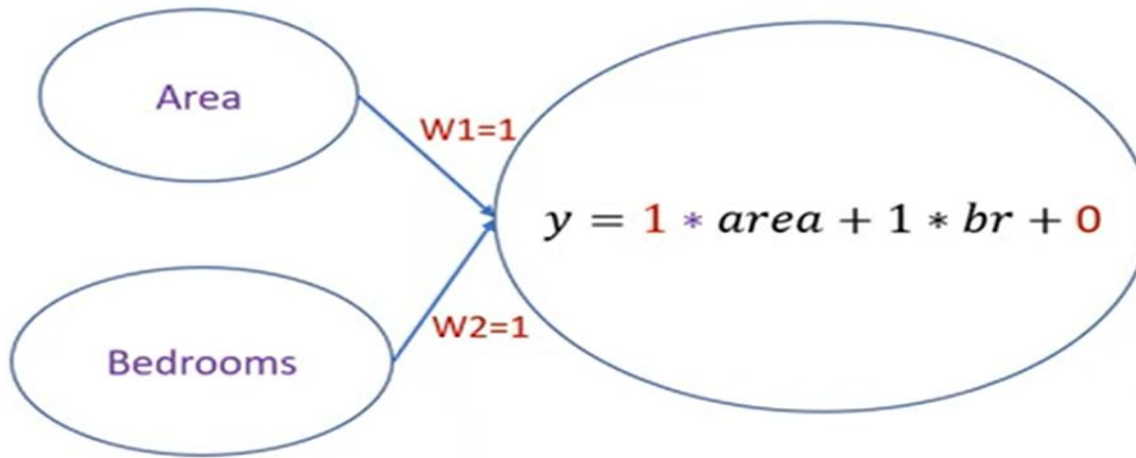
$$w1 = w1 - \textit{something}$$

$$w1 = w1 - \textit{learning rate} * \partial / \partial w1$$

$$w2 = w2 - \textit{learning rate} * \partial / \partial w2$$

$$b = b - \textit{learning rate} * \partial / \partial b$$

Chain Rule



$$w1 = w1 - learning\ rate * \frac{\partial}{\partial w1}$$

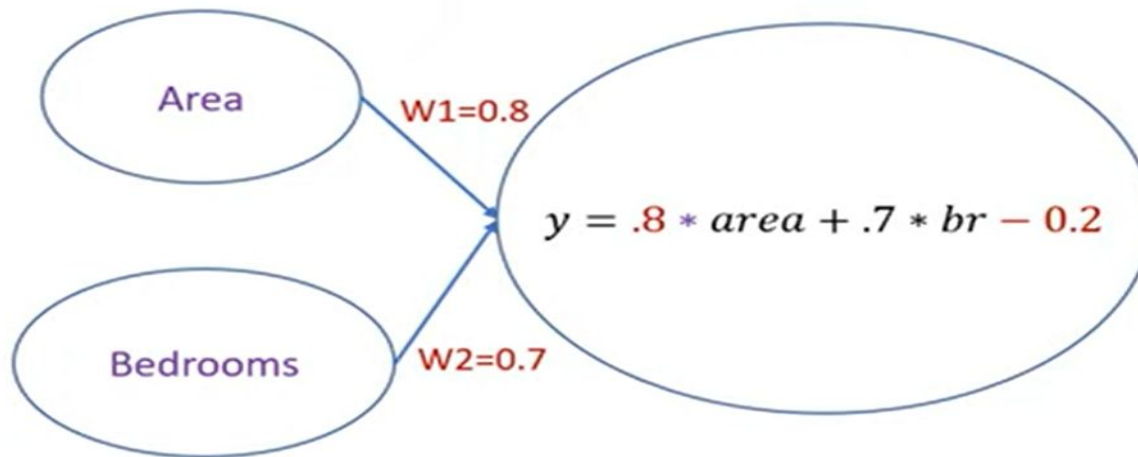
$$w1 = 1 - 0.2 = 0.8$$

$$w2 = w2 - learning\ rate * \frac{\partial}{\partial w2}$$

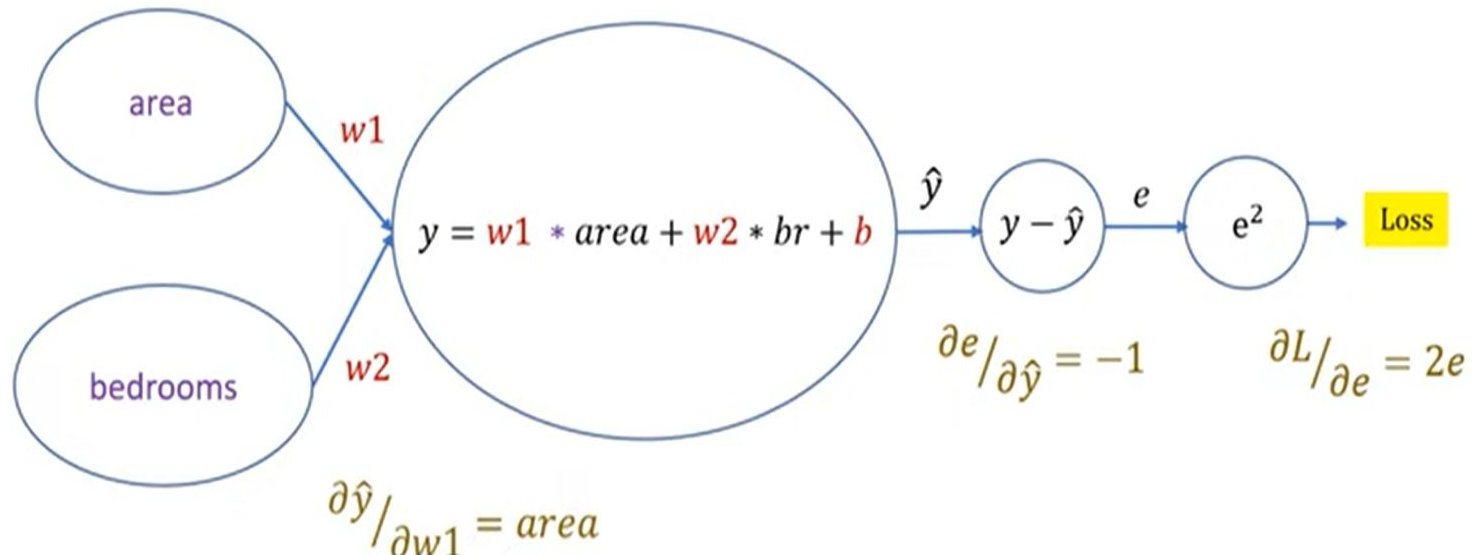
$$w2 = 1 - 0.3 = 0.7$$

$$b = b - learning\ rate * \frac{\partial}{\partial b}$$

$$b = 0 - 0.2 = -0.2$$



Chain Rule



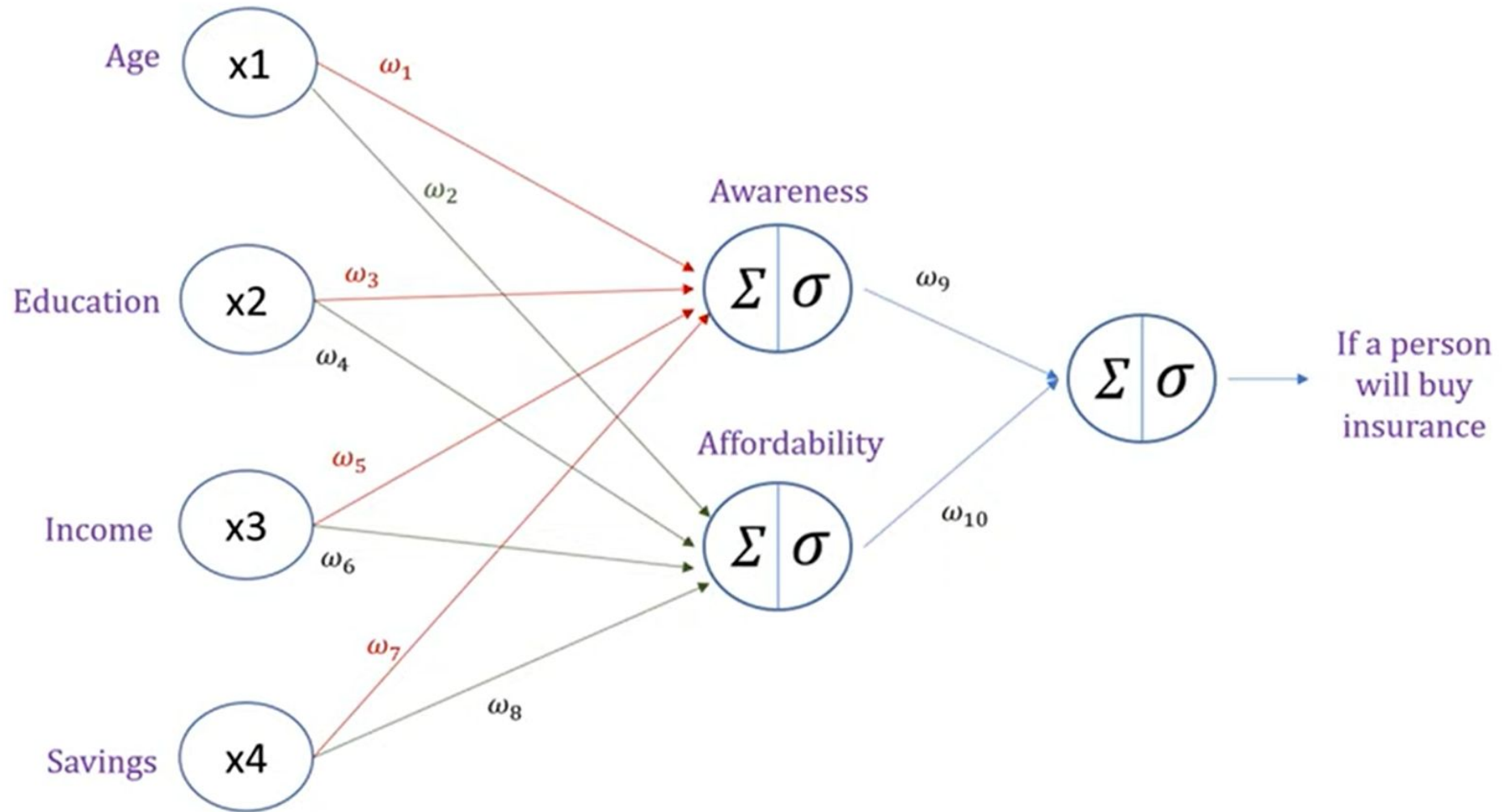
$$\frac{\partial L}{\partial w_1} = ?$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial e} * \frac{\partial e}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_1}$$

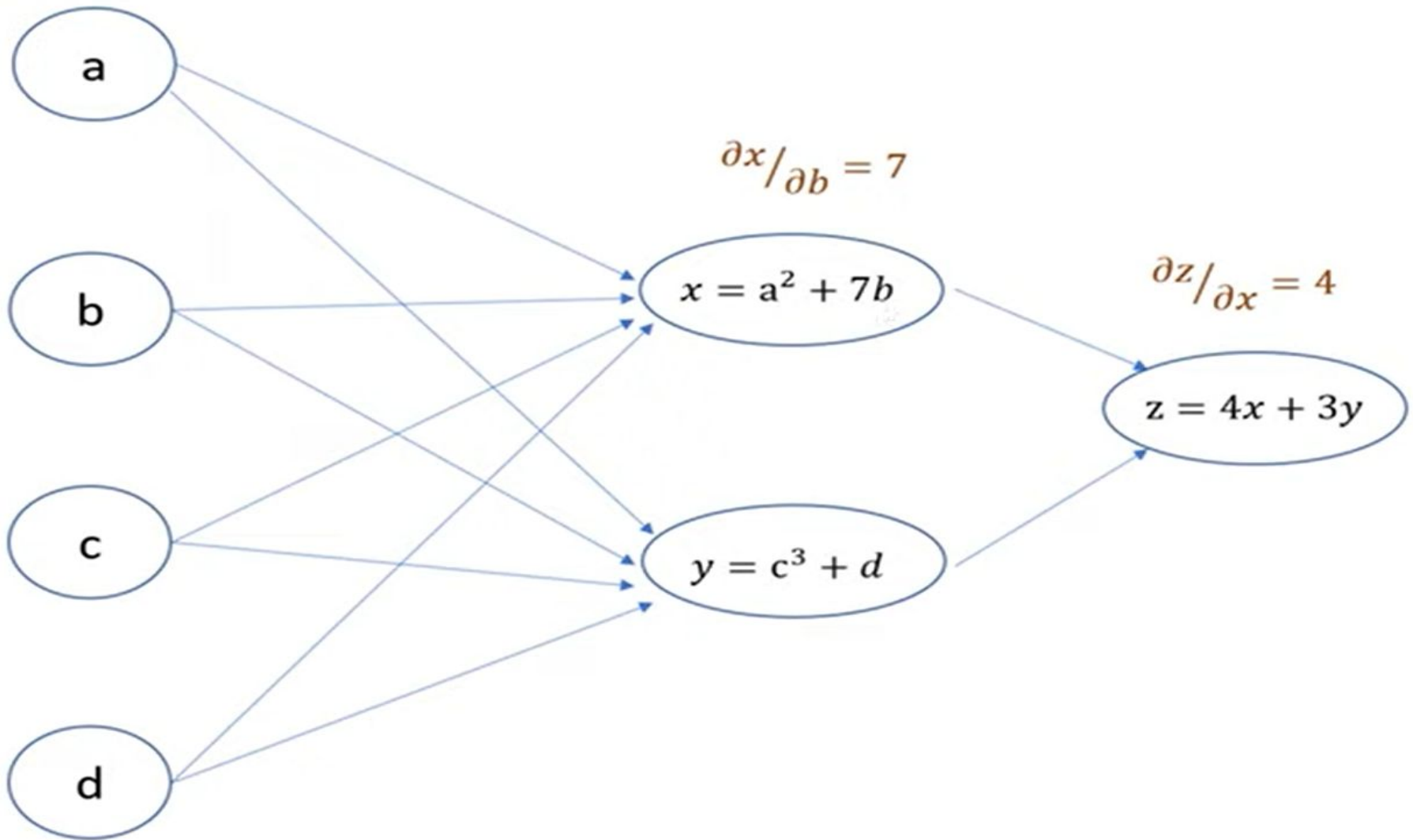
$$\frac{\partial L}{\partial w_1} = -2e * \text{area}$$

Chain Rule

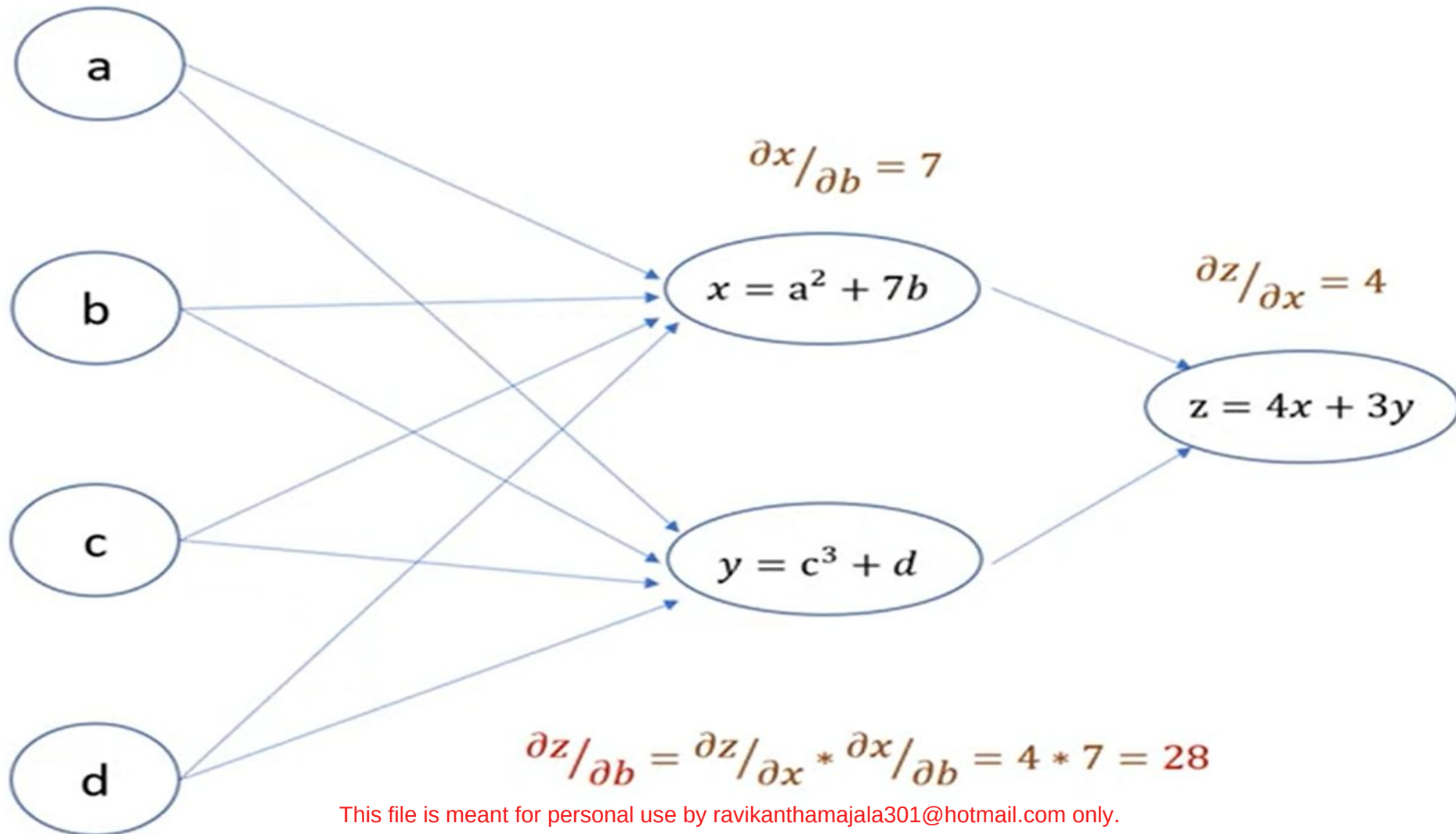
Chain Rule



Chain Rule



Chain Rule



Binary Cross Entropy Loss

Binary Cross Entropy Loss

Cross entropy

- **Cross-entropy** builds upon the idea of information theory entropy, and measures the **difference** between **two probability distributions** for a given random variable / set of events.

Types:

1. **Categorical cross-entropy** - multi-class classification $L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(y'_i)$
2. **Binary cross-entropy** – Binary class classification

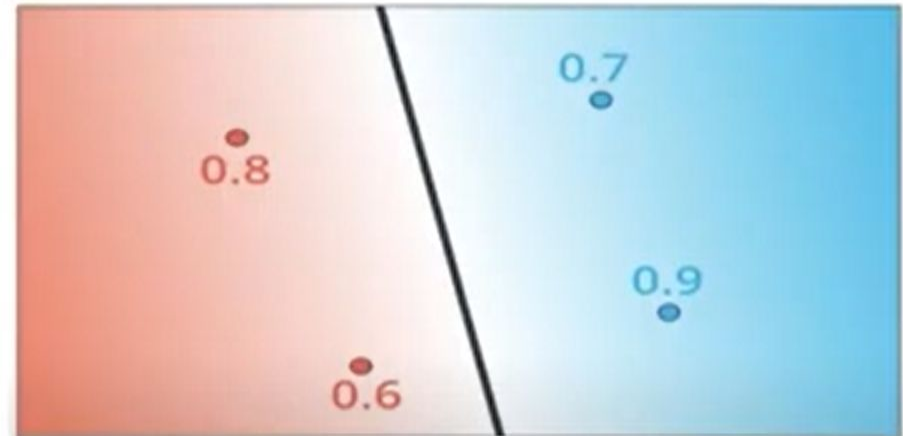
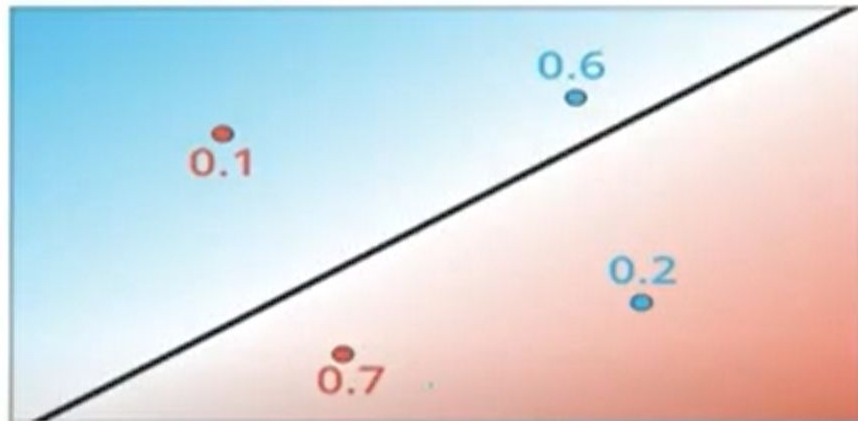
Binary cross-entropy is a loss function that is used in binary classification problems.

$$BCE = -\frac{1}{m} \sum_{i=1}^m (y_i \cdot \log(y'_i) + (1 - y_i) \cdot \log(1 - y'_i))$$

- m - the number of data points.
- y_i - the actual label of the data point. Class label be 0 or 1.
- y'_i - the predicted probability of the data point. Between 0 and 1.

Binary Cross Entropy Loss

Why ***-log*** is used in cross entropy? Compare the Model A and Model B



Product probability: *The probability of two (or more) independent events that are occurring together is calculated by multiplying the event's individual probabilities.*

- Calculate the total probability of the models by multiplying the probability of each independent sample.

- **Product Probability Model A:**

$$0.1 * 0.7 * 0.6 * 0.2 = 0.0084$$

- **Product Probability Model B:**

$$0.8 * 0.6 * 0.7 * 0.9 = 0.3024$$

- The product probability for model B is **better** (max value) than model A.

This file is meant for personal use by ravikanthamajala301@hotmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.

Binary Cross Entropy Loss

Why **-log** used in cross entropy? Compare the Model A and Model B

- *Product probability* works better for **dataset with less samples**, but **not possible** with real-life **large dataset**.
- E.g., Assume the dataset with 1000 students, the product probabilities will always be closer to 0, even for efficient DL model.
- If change one probability, the product will change drastically, and give the wrong decision that a model performs well. So, we need to **transform** the **products** to a **sum** using a **logarithmic function**.

$$0.1 * 0.7 * 0.6 * 0.2 = 0.0084$$

- **Log Model A:**

$$\begin{aligned} &\log(0.1) + \log(0.7) + \log(0.6) + \log(0.2) \\ &-1 + -0.154 + -0.221 + -0.698 = -2.073 \end{aligned}$$

- **Log Model B:**

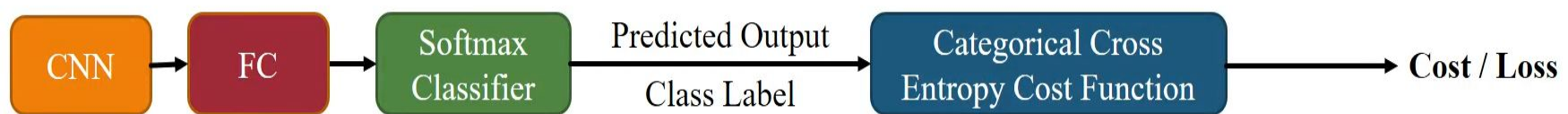
$$\begin{aligned} &\log(0.8) + \log(0.6) + \log(0.7) + \log(0.9) \\ &-0.09 + -0.22 + -0.15 + -0.045 = -0.505 \end{aligned}$$

Categorical Cross Entropy Loss

Categorical Cross Entropy Loss

Categorical (Multi-class) cross-entropy

- Categorical (Multi-class) cross-entropy is also known as Softmax Loss .
- It is used for multi-class classification with two or more output classes (labels).



- The sum of probabilities of each class is 1.
- The output label is assigned **one-hot category encoding** value.
- If output label is in **integer** form which is converted into categorical encoding using `keras.utils.to_categorical` method.
- For multi-class classification problem, calculate the Cost using **Log loss** $L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(y'_i)$
- **Code:**

```
model.compile(optimizer=optimizers.SGD(lr=0.01), loss='categorical_crossentropy')
```

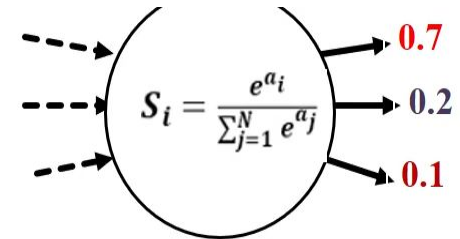
This file is meant for personal use by ravikanthamajala301@hotmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.

Categorical Cross Entropy Loss

Multi class classifier activation function- Softmax

$$\text{Softmax}_i = \frac{e^{a_i}}{\sum_{j=1}^N e^{a_j}} \quad \forall j \in 1..N$$



Ex: 2 Consider the input to the softmax function is $\{2, 1, 0.1\}$, compute the output of the given input.

$$S(a_1) = \frac{e^{a_1}}{e^{a_1} + e^{a_2} + e^{a_3}} = \frac{e^2}{e^2 + e^1 + e^{0.1}} = 0.7$$

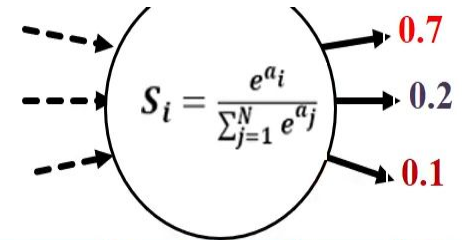
$$S(a_2) = \frac{e^{a_2}}{e^{a_1} + e^{a_2} + e^{a_3}} = \frac{e^1}{e^2 + e^1 + e^{0.1}} = 0.2$$

$$S(a_3) = \frac{e^{a_3}}{e^{a_1} + e^{a_2} + e^{a_3}} = \frac{e^{0.1}}{e^2 + e^1 + e^{0.1}} = 0.1$$

It results 0.7 as output i.e., maximum likelihood.

	x_1	x_2	x_3	y	Equivalent y	y'	Classes
S_1	2	1	0.1	Orange	1	0.7	Orange
					0	0.2	Apple
					0	0.1	Lemon
S_2	0	1	1	Lemon	0	0.3	Orange
					0	0.4	Apple
					1	0.3	Lemon
S_3	1	0	0	Lemon	0	0.1	Orange
					0	0.5	Apple
					1	0.4	Lemon

Categorical Cross Entropy Loss



Multi class classifier activation function- Softmax

Assume that predicted probabilities for three samples is 0.7, 0.3, 0.4.

Find the Cost of this dataset.

Product probabilities for predicted values = $0.7 * 0.3 * 0.4 = 0.084$

Cross Entropy = *sum of the negative logarithm of predicted probabilities of each sample.*

$$= -\log(0.7) + (-\log(0.3)) + (-\log(0.4)) = 1.073$$

Loss for Sample 1: $L = -y_i \cdot \log(y'_i)$

$$= -(1 * \log(0.7)) + (-[0 * \log(0.2)]) + (-[0 * \log(0.4)])$$

$$= -(1 * (-0.35667)) + 0 + 0 = -(-0.35667) = 0.35667$$

	x_1	x_2	y	Equivalent y	y'	Classes
S_1	0	0	Orange	1	0.7	Orange
				0	0.2	Apple
				0	0.1	Lemon
S_2	0	1	Lemon	0	0.3	Orange
				0	0.4	Apple
				1	0.3	Lemon
S_3	1	0	Lemon	0	0.1	Orange
				0	0.5	Apple
				1	0.4	Lemon

Loss for Sample 2: $L = -y_i \cdot \log(y'_i) = -(0 * \log(0.3)) + (-[0 * \log(0.4)]) + (-[1 * \log(0.3)])$

$$= -(0 + 0 + (1 * [-1.20397])) = -(-1.20397) = 1.20397$$

Loss for Sample 3: $L = -y_i \cdot \log(y'_i) = -(0 * \log(0.1)) + (-[0 * \log(0.5)]) + (-[1 * \log(0.4)])$

$$= -(0 + 0 + (1 * [-0.91629])) = -(-0.91629) = 0.91629$$

$$\text{Cross Entropy } J = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(y'_i) = -\frac{1}{3} * [0.35667 + 1.20397 + 0.91629] = -(0.33 * 2.47693) = 0.81738$$

This file is meant for personal use by ravikanthamajala301@hotmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.